



Continuous Integration for Hardware

GitHub Actions • Gerber automation • ERC/DRC • Release pipelines

Software-style CI for KiCad projects

Audience: Engineering Students & Junior Hardware Engineers

Why • Tools • Workflows • Releases • Quality



Why CI for Hardware?

Software has CI for 20 years — hardware is catching up

Auto Gerber

Every commit produces a ready-to-fab fileset.

ERC / DRC

Caught at commit time, not review time.

Auto BOM

Always-current BOM for procurement.

3D render

Visual diffs of schematic & PCB changes.

Releases

Tag a version → complete fab package generated.

Quality gate

Block merges with errors — no bad PCBs.

CI Tools for KiCad

Open-source ecosystem in 2026

KiBot

Best CI tool for KiCad.
YAML config, lots of outputs.

INTI-CMNB KiAuto

Generates Gerber, schematic
PDF, 3D renders headless.

kiCad-cli

Built into KiCad 8+.
Native export commands.

GitHub Actions

Free for public repos.
Docker containers run KiCad.

GitLab CI

Self-hostable runner,
same YAML idea.

kiCad-diff

Visual schematic +
PCB diff in PRs.

What Can be Automated

Every output of your KiCad project

- ▶ Gerber + Excellon drill files (RS-274X)
- ▶ Pick-and-place (.pos) for SMT assembly
- ▶ Bill of Materials (BOM) — CSV with manufacturer + parts
- ▶ Schematic PDF — for review and documentation
- ▶ 3D render (PNG / GIF) — visual aid
- ▶ STEP / VRML — for mechanical CAD
- ▶ ERC and DRC reports — pass/fail summary
- ▶ JLCPCB / PCBWay-format ZIP bundles

Sample GitHub Actions Workflow

Build Gerbers on every commit

```
# .github/workflows/build.yml
name: Build PCB Outputs
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    container:
      image: setsoft/kicad_auto:dev_k8
    steps:
      - uses: actions/checkout@v4

      - name: Run KiBot
        run: kobot -c .kobot.yaml -d output

      - name: Upload Gerbers
        uses: actions/upload-artifact@v4
        with:
          name: gerbers
          path: output/gerber/

      - name: Upload BOM
        uses: actions/upload-artifact@v4
        with:
          name: bom
```

Sample KiBot Config

What to generate, in YAML

```
# .kibot.yaml
kibot:
  version: 1

preflight:
  run_erc: true
  run_drc: true
  update_xml: true
  check_zone_fills: true

outputs:
  - name: gerbers
    type: gerber
    dir: gerber/
  - name: drill
    type: excellon
    dir: gerber/
  - name: bom
    type: bom
    dir: bom/
    options:
      format: HTML
  - name: position
    type: position
    dir: gerber/
  - name: 3d
    type: render_3d
    dir: 3d/
  - name: schematic
    type: pdf_sch_print
    dir: docs/
```

Pull-Request Workflow

Quality gates before merging

- ▶ Designer pushes change to feature branch
- ▶ GitHub Actions runs: ERC + DRC + Gerber + schematic PDF
- ▶ Failed ERC/DRC blocks merge automatically
- ▶ kicad-diff posts visual diff as PR comment
- ▶ Schematic PDF attached as artifact — reviewer opens directly
- ▶ 3D render embedded — reviewer sees what changed
- ▶ Once approved, merge into main
- ▶ Tag the release → fab package auto-attached to GitHub Release page

Release Pipeline

Tag a commit → fab fileset published

```
# .github/workflows/release.yml
name: Release Build
on:
  push:
    tags: ['v*']

jobs:
  release:
    runs-on: ubuntu-latest
    container:
      image: setsoft/kicad_auto:dev_k8
    steps:
      - uses: actions/checkout@v4
      - name: Generate fab package
        run: kobot -c .kobot.yaml -d output

      - name: Zip artifacts
        run: |
          cd output
          zip -r fab-package.zip .

      - name: Create GitHub Release
        uses: softprops/action-gh-release@v2
        with:
          files: output/fab-package.zip
          generate_release_notes: true
```

Beyond Gerber — Other CI Possibilities

What else can be automated

Firmware CI

Compile, unit-test
FW for the PCB.

Hardware-in-loop

Connect dev-board
to runner — test live.

3D model export

STEP / VRML for
mech engineers.

Cost analysis

OctoPart API: compute
BOM cost on every commit.

Component lifecycle

Flag EOL parts —
warn before redesign.

Sim runs

Run SPICE batch tests
on every commit.

CI Best Practices

How to use CI without going overboard

- ▶ Start with ERC + DRC + Gerber only
 - ▶ Add more outputs as the project grows
 - ▶ Keep build time < 5 min for fast feedback
 - ▶ Cache KiCad libraries between runs
 - ▶ Fail loud — clear error messages
 - ▶ Pin tool versions to avoid surprise breakage
- ▶ Use Docker images — reproducible builds
 - ▶ Test the build locally before pushing CI changes
 - ▶ Make CI output discoverable (GitHub artifacts page)
 - ▶ Auto-tag releases off main branch
 - ▶ Keep secrets out of YAML (use Actions Secrets)
 - ▶ Document the CI in the repo's README

Common CI Issues

Hardware CI is younger than software — bumps to expect

- ▶ KiCad version mismatch — pin via Docker tag
- ▶ Custom library paths broken in CI — make paths relative
- ▶ Large 3D models slow the runner — use LFS or skip 3D in CI
- ▶ Plugin dependencies — install in Dockerfile or per-step
- ▶ Stale fp-info-cache — rebuild explicitly
- ▶ Symbols renamed across rev — update KiBot config
- ▶ Big repos hit free-tier minute limits — self-host runner
- ▶ Errors only visible in artifacts — surface them in PR comments

Tools Tour

Specific projects to bookmark

KiBot

Most popular CI tool.
github.com/INTI-CMNB/KiBot

KiAuto

Underlying scripts that
KiBot wraps.

kiCad-diff

Schematic & PCB visual
comparison tool.

kiCad-make

Lightweight make-based
workflow alternative.

OctoPart API

BOM cost & lifecycle
automation.

KiCanvas

Web-based KiCad viewer —
embed in PR comments.

Self-Hosted Alternatives

When GitHub Actions isn't an option

- ▶ GitLab CI — self-hostable runner, same YAML style
- ▶ Jenkins — old-school but powerful
- ▶ Drone CI — lightweight, Docker-friendly
- ▶ Buildkite — distributed runner, mid-size organisations
- ▶ Local make + scripts — simplest, no infrastructure
- ▶ GitHub free tier: 2k minutes / month — usually enough for hardware projects
- ▶ Self-hosted runner: Raspberry Pi or old laptop can run the jobs
- ▶ Privacy: self-host for confidential commercial projects

Key Takeaways

Key takeaways from this lecture

Hardware CI is here

KiCad CI tools are mature.

Start with ERC/DRC

Quick win, big impact.

KiBot + GitHub Actions

Best free combo today.

Automate releases

Tag → fab fileset.

Visual diffs save reviews

kicad-diff in PRs.

Keep it fast

< 5 min build cycle.

Questions & Discussion

Ask anything — concepts, projects, sourcing, career.

References & Further Learning

- ▶ KiBot — github.com/INTI-CMNB/KiBot
- ▶ kicad-cli docs.kicad.org
- ▶ kicad-diff — [leoheck/kicad-diff](https://github.com/leoheck/kicad-diff)
- ▶ GitHub Actions documentation
- ▶ GitLab CI documentation
- ▶ Robert Feranec / Phil's Lab — CI for KiCad videos
- ▶ OctoPart API for BOM automation
- ▶ KiCanvas — embed KiCad rendering in web

Thank you • Build something this week.